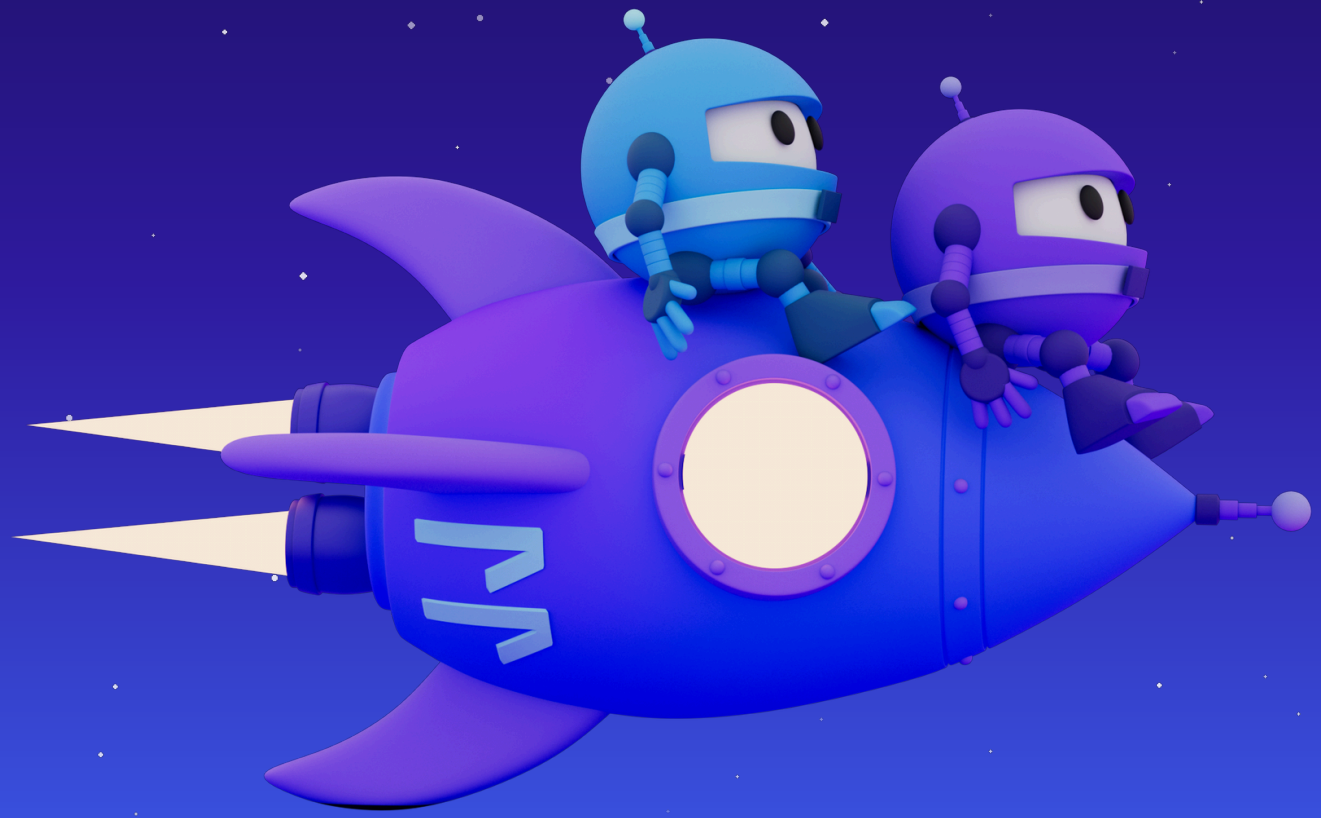
Building Dual-Screen Experiences with Avalonia

.NET

Unfold your UI on iPhone Duo,
Android and desktop



Javier Suárez

Software Engineer, Avalonia UI · @jsuarezruiz



About me

Javier Suárez

Software Engineer at Avalonia UI, building cross-platform UI.

Before Avalonia

Worked on Xamarin.Forms and .NET MAUI at Microsoft

Community

Former Microsoft MVP, speaker, and one of the people behind SevillaDotNet

OSS

Avalonia: Nova.Avalonia.UI, Cupertino.Avalonia, Vignettes
.NET MAUI: awesome-dotnet-maui (2.7k stars), AlohaKit
Xamarin.Forms: goodlooking-UI (2.5k stars), Xamanimation

Find me

@jsuarezruiz on GitHub, X and Mastodon
@javiersuarezruiz on YouTube

Avalonia in 30 seconds

Before we start

31.8k

GitHub stars, open source under the MIT license

github.com/AvaloniaUI/Avalonia, Nov 2026

29.6M

NuGet downloads of the Avalonia package

nuget.org, Nov 2026

6

Windows, macOS, Linux, iOS, Android and the browser

One codebase, Avalonia 12

C# and XAML, close to WPF. Avalonia draws every pixel itself with Skia, so a layout looks and measures the same on every platform, folds included.

Foldables aren’t new

Why now

2018	2019	2020	2022	2023	2026
Royole FlexPai	**Galaxy Fold**	**Surface Duo**	**Android 12L**	**Pixel Fold**	**iPhone Duo**
The first foldable phone on sale.	Samsung takes book foldables mainstream.	Microsoft’s two screens and a hinge, with TwoPaneView.	Android designs for large screens and folds.	Google’s own book foldable.	Apple’s first foldable.

Friday, October 23, 2026

The iPhone folds.

iPhone Duo: a 7.6" inner display, a 5.4" outer display, and a hinge built from more than 100 components. From $1,999.

“iPhone Duo is the most transformational change to iPhone since the original.”

John Ternus, Apple’s CEO



Source: Apple Newsroom, “Apple unveils iPhone Duo”, September 2026

Small market, big spenders, and now Apple

Why now

18.2M

foldable phones shipped worldwide in 2025

Omdia, Aug 2026

20M+

forecast to ship in 2026, iPhone Duo included

TrendForce 20.2M, Omdia 22M+

580M+

large-screen Android devices in use

Google, May 2026

~25%

Apple’s 2026 share, about 5M iPhone Duos

TrendForce forecast, Sep 2026

40%

Apple’s expected share of all foldables in 2027

IDC and Counterpoint forecasts

up to 14×

how much more foldable users can spend than phone-only users

Google internal data, 2026

The platforms stopped asking

Why now

Android 17 · API 37

Orientation locks are ignored on large screens

On sw600dp displays and up, the system ignores screenOrientation, resizeableActivity and aspect-ratio limits. Targeting API 36 you can opt out. Targeting API 37 you can't.

iPhone Duo · iOS 27.1 SDK

Edge to edge on the inner display

Apps built with the iOS 27.1 SDK or later fill the inner display, bars move to the side, and supported orientations are ignored there.

"On iPhone Duo, the device's orientation doesn't tell you what shape your app is." Apple, developer.apple.com/iphone-duo/prepare

Four postures to design for

Part 1 of 4 · Core concepts

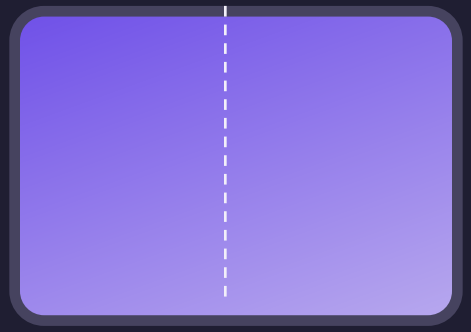
Book foldables (iPhone Duo, Galaxy Z Fold, Pixel Fold) and flips (Galaxy Z Flip, razr) share them, and your app moves between displays while it runs.



Closed

`Closed`

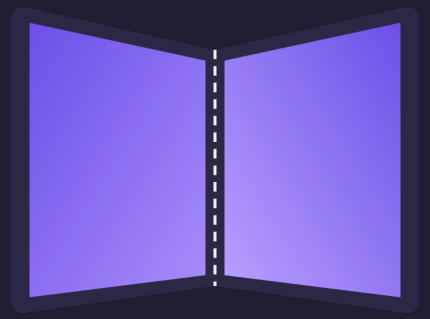
The cover display. Usually one pane.



Flat

`Flat`

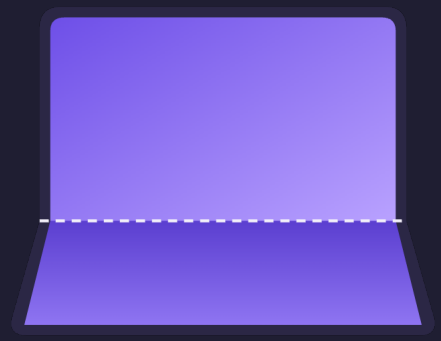
Fully open. The fold is there, but it divides nothing.



Book

`HalfOpened + Vertical`

Held like a book. Two pages, left and right.



Tabletop

`HalfOpened + Horizontal`

Standing on a table. Content above, controls below.

The fold is just a rectangle

Core concepts



iPhone Duo inner display: 951 × 669pt, fold at x = 455.5

`FoldBounds`
Where the fold is, in window coordinates. Nothing tappable goes there.

`IsSeparating`
Does it divide the content right now? True when half open.

`Orientation`
Vertical: panes left and right. Horizontal: panes above and below.

One idea, three APIs

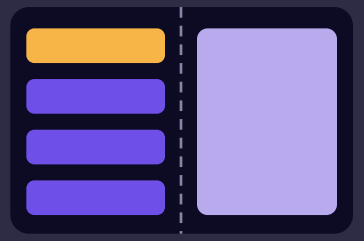
Core concepts

Concept	Android · WindowManager	iOS 27.1 · UIKit	Avalonia · Nova
Posture	FoldingFeature.state	UIHinge.status	Fold.Posture
Where	FoldingFeature.bounds	reservedRegions(kind: .division)	FoldState.FoldBounds
Divides?	FoldingFeature.isSeparating	division region active	Fold.IsSeparating
Direction	FoldingFeature.orientation	region frame shape	Fold.Orientation
Hinge angle	TYPE_HINGE_ANGLE sensor	UIHinge.angle	Fold.HingeAngle
Space	WindowSizeClass, 600/840dp	Size classes	TwoPaneView, 641

Same concepts, different names. Your shared code should only ever see the right-hand column.

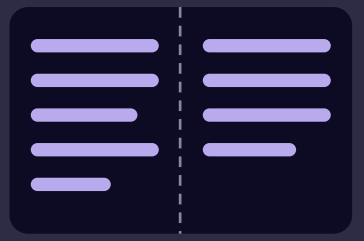
Five layouts that use the fold

Core concepts



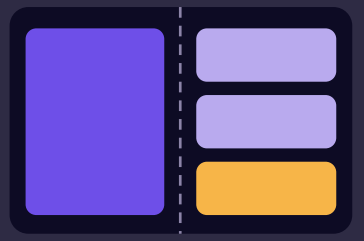
List and detail

The list on one side, the selected item on the other. Mail, notes, settings.



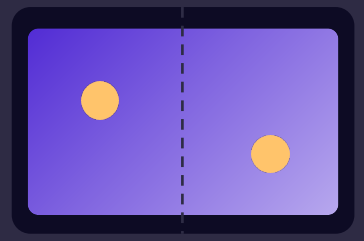
Two pages

Content flows across both sides like a book. Readers, docs, calendars.



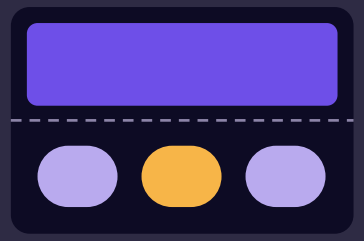
Companion pane

The work on one side, its tools on the other. Editors, maps with directions.



Extended canvas

One surface across the fold, with nothing important on the crease. Maps, photos.



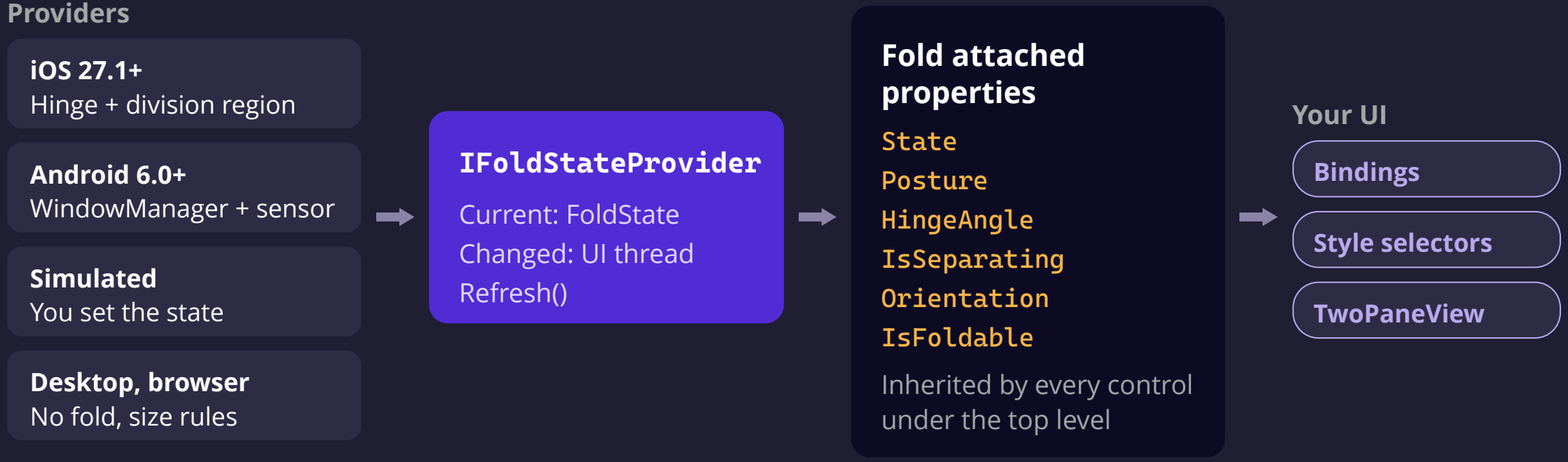
Tabletop

Content above the fold, controls below it. Video, camera, calls.

Meet Nova.Avalonia.Foldable

Part 2 of 4 · The library

The fold as Avalonia properties, plus a TwoPaneView. One shared codebase, from the device to your XAML.



One small change per app head

The library

Android · MainActivity.cs

```
[Activity(MainLauncher = true,
    ResizeableActivity = true,
    ConfigurationChanges =
        ConfigChanges.Orientation
      | ConfigChanges.ScreenSize
      | ConfigChanges.UiMode
      | ConfigChanges.ScreenLayout
      | ConfigChanges.SmallestScreenSize
      | ConfigChanges.Density)]
public class MainActivity
    : AvaloniaMainActivity { }
```

Folding is a configuration change. Handle it, and the activity isn't recreated.

iOS · the app head's .csproj

```
<PropertyGroup>
  <TargetFramework>net10.0-ios27.1</TargetFramework>
</PropertyGroup>

<!— Older iOS still runs, without a fold —>
```

The hinge APIs ship in .NET for iOS 27.1. Keep SupportedOSPlatformVersion as it is: older iOS still runs.

Read the fold in XAML

Part 3 of 4 · Sample 01

```xaml
<UserControl xmlns="https://github.com/avaloniaui"
             xmlns:fold="using:Nova.Avalonia.Foldable"
             fold:Fold.IsEnabled="True">
  <UserControl.Styles>
    <!-- The fold divides the view: an even number of columns -->
    <Style Selector="UniformGrid.photos[(fold|Fold.IsSeparating)=True]">
      <Setter Property="Columns" Value="4" />
    </Style>
    <!-- Half open: dim the chrome a little -->
    <Style Selector="Border.controls[(fold|Fold.Posture)=HalfOpened]">
      <Setter Property="Opacity" Value="0.85" />
    </Style>
  </UserControl.Styles>
  <TextBlock Text="{Binding $self.(fold:Fold.Posture)}" />
</UserControl>
```

What the app sees

IsFoldable	True
Posture	HalfOpened
HingeAngle	110°
IsSeparating	True
Orientation	Vertical
FoldBounds	455.5,0 40×669

Book posture on the iPhone Duo. Styles re-apply on their own when the fold changes. Selectors use `|`, bindings use `:`

List and detail with TwoPaneView

Sample 02

```
<fold:TwoPaneView PanePriority="Pane1"
                  MinWideModeWidth="641"
                  MinTallModeHeight="100000">
  <fold:TwoPaneView.Pane1>
    <local:InboxView />
  </fold:TwoPaneView.Pane1>
  <fold:TwoPaneView.Pane2>
    <local:MessageView />
  </fold:TwoPaneView.Pane2>
</fold:TwoPaneView>
```

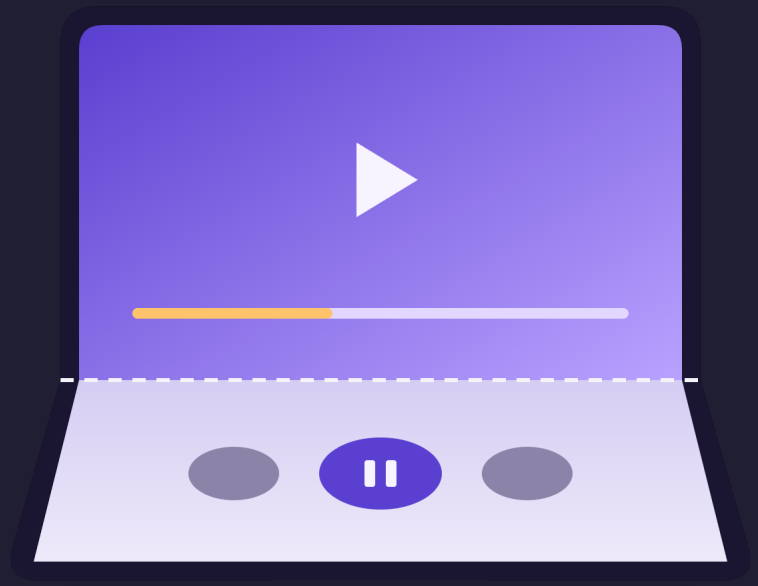
The WinUI and .NET MAUI TwoPaneView API, so existing code carries over.

How it picks a layout

1. The fold divides the view: a pane on each side, never on the fold.
2. A flat fold crosses a wide view: split at the fold, with SnapToFold.
3. Wider than MinWideModeWidth: side by side.
4. Taller than MinTallModeHeight: stacked.
5. Otherwise one pane: the one PanePriority names.

Tabletop: content up, controls down

Sample 03



Pane 2 above the fold, Pane 1 below it

```
<fold:TwoPaneView PanePriority="Pane2"
                  MinWideModeWidth="700"
                  MinTallModeHeight="100000"
                  TallModeConfiguration="BottomTop">
  <fold:TwoPaneView.Pane1>
    <local:PlayerControls />
  </fold:TwoPaneView.Pane1>
  <fold:TwoPaneView.Pane2>
    <local:VideoView />
  </fold:TwoPaneView.Pane2>
</fold:TwoPaneView>
```

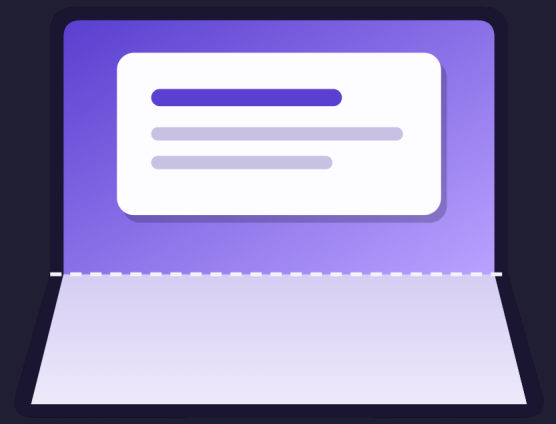
A huge MinTallModeHeight stops a tall phone from stacking the panes. A dividing horizontal fold still does, and BottomTop puts Pane1 under it, near the thumbs.

Place things around the fold

Sample 04

```
protected override void OnPropertyChanged(
    AvaloniaPropertyChangedEventArgs change)
{
    base.OnPropertyChanged(change);
    if (change.Property ≠ Fold.StateProperty) return;

    if (Fold.GetState(this) is { IsTabletop: true, FoldBounds: { } fold }
        && TopLevel.GetTopLevel(this) is { } top
        && top.TransformToVisual(this) is { } toLocal)
    {
        // FoldBounds is in top-level coordinates: map it first
        var local = fold.TransformToAABB(toLocal);
        Dialog.MaxHeight = local.Top - 16; // stay above the crease
    }
}
```



In tabletop the dialog stays in the top half, clear of the fold

The hinge as an input

Sample 05



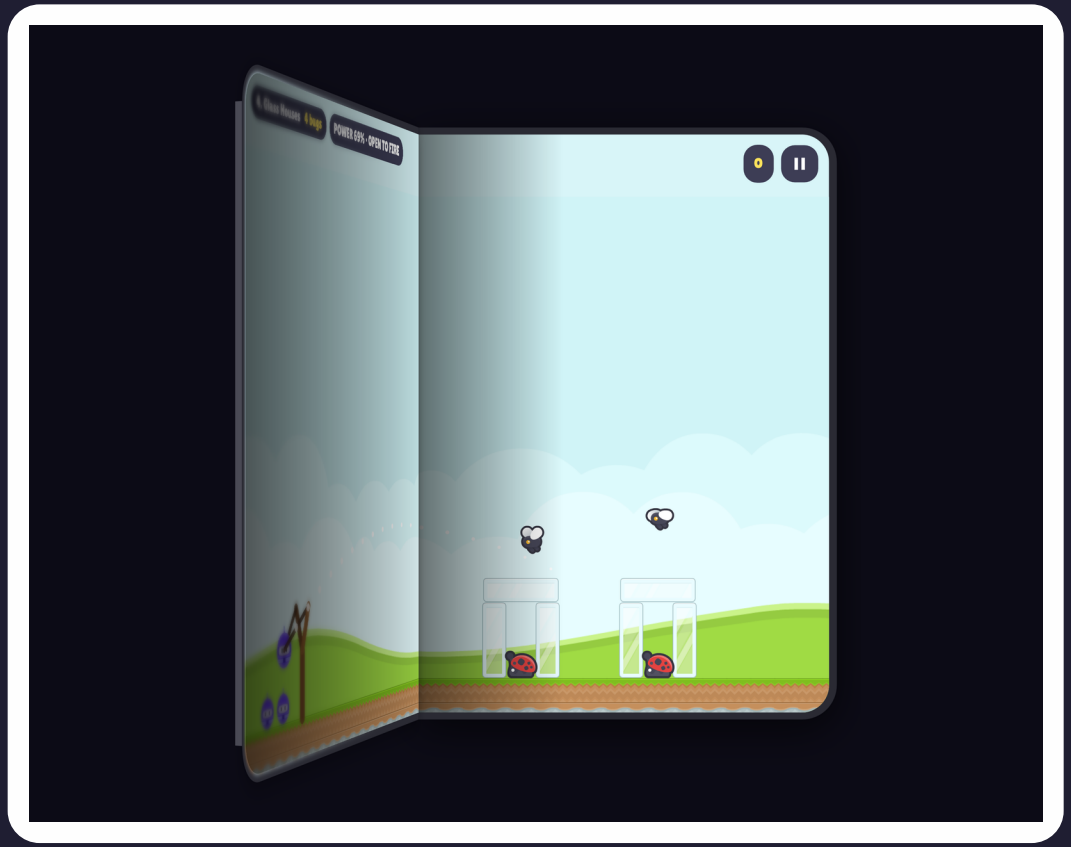

```
protected override void OnPropertyChanged(
    AvaloniaPropertyChangedEventArgs change)
{
    base.OnPropertyChanged(change);

    if (change.Property == Fold.HingeAngleProperty
        && change.OldValue is double oldAngle
        && change.NewValue is double newAngle)
    {
        // Folding drives the stars, the way scrolling does
        DriveStars((newAngle - oldAngle) * HingeWarp);
    }
}
```

At most one update per frame. Effects only: the hinge sensor is optional on Android.

Bot vs Bugs, the hinge as a slingshot

Sample 06

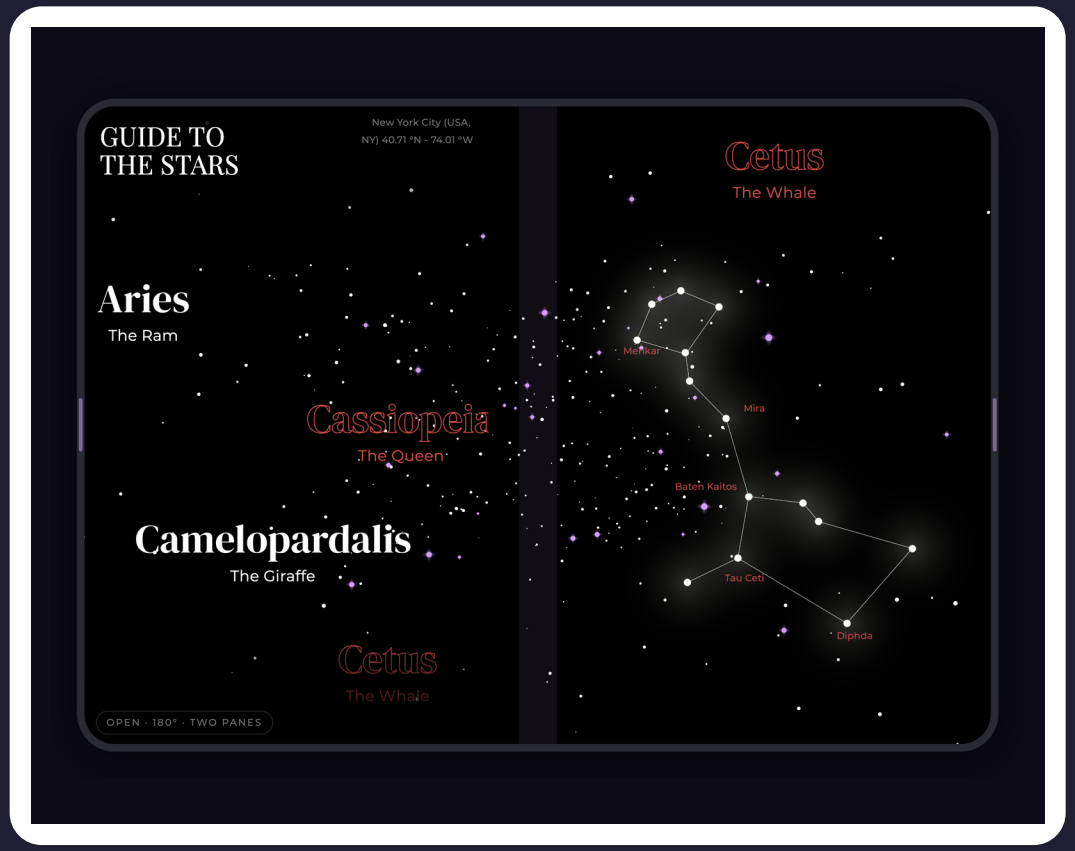


- Fold the phone to pull the sling, open it to fire
- The deeper the fold, the stronger the shot, from Fold.HingeAngle
- Flat or book: one level across both halves
- Tabletop: the game above, a control deck below
- Closed: it keeps playing on the cover display

Physics by Aether.Physics2D, art by Kenney, the bot from the .NET brand

Guide To the Stars, on iPhone Duo

Sample 07

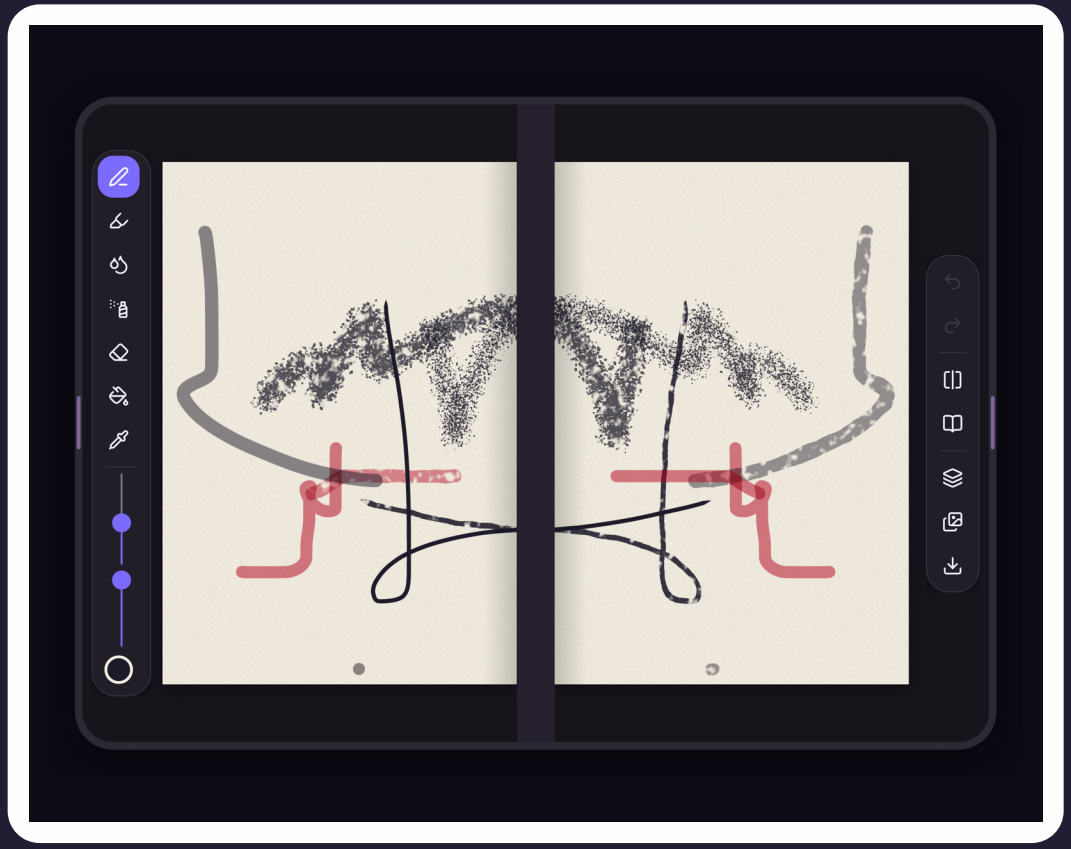


- List and detail on either side of the fold, under one sky
- The title card flies across the fold when you pick a constellation
- A stacked layout in tabletop posture
- State kept when the device folds and unfolds
- The hinge warps the stars, the way scrolling does

The “Guide To the Stars” vignette from Avalonia Vignettes, adapted to two screens

Inkfold, a sketchbook that folds

Sample 08

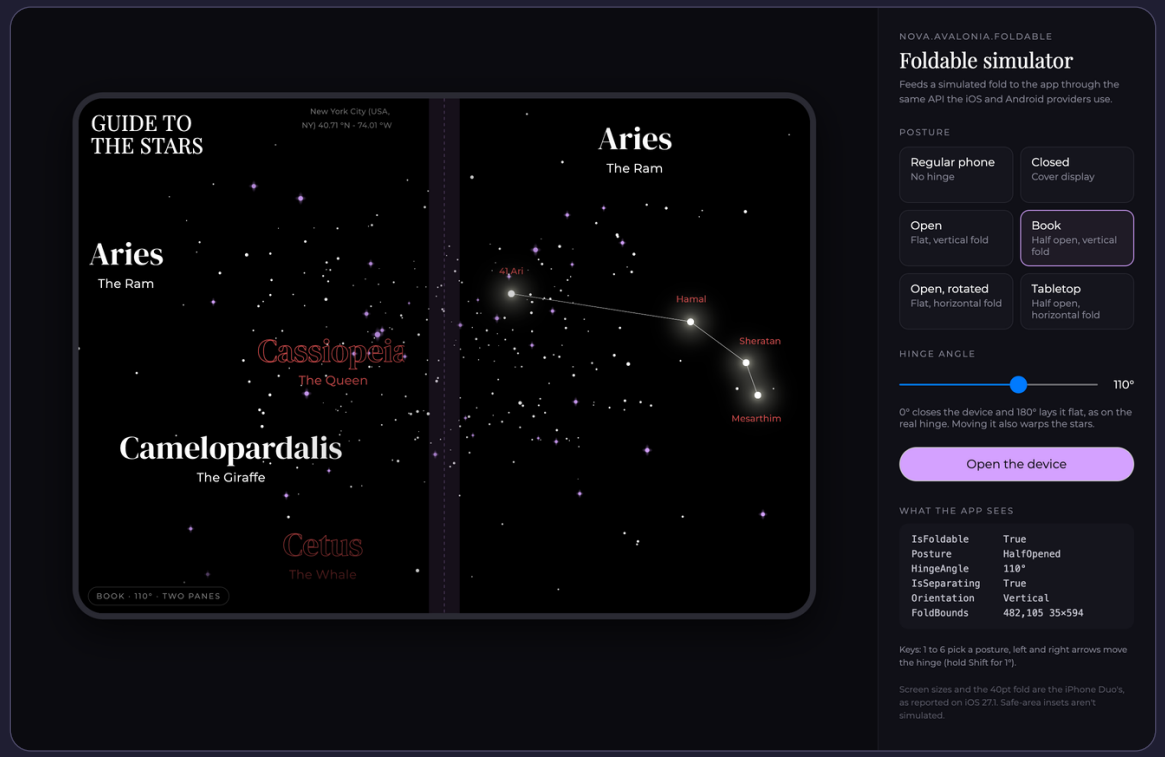


- The page opens across the fold like a book
- Arm the press, close the phone and open it: the paint prints on the facing page
- Tabletop: page above, brushes and layers below
- Pen, marker, watercolor and spray, in SkiaSharp
- Layers, undo, symmetry, zoom and a gallery

The apps and these slides: jsuarezruiz/dotnetconf2026-avalonia-foldables

No foldable? Simulate one

Sample 09




```
var fold = new SimulatedFoldStateProvider();
Fold.SetProvider(window, fold);

fold.Current = new FoldState(
  IsFoldable: true,
  Posture: DevicePosture.HalfOpened,
  HingeAngle: 110,
  FoldBounds: new Rect(380, 0, 40, 860),
  IsSeparating: true);
```

Same API as the real providers. Keys 1 to 6 pick a posture, the arrows move the hinge.

Unit-test fold layouts, headless

Sample 10

```
[AvaloniaFact]
public void Half_Open_Splits_At_The_Fold()
{
    var view = new TwoPaneView { Pane1 = new Border(), Pane2 = new Border() };
    var window = new Window { Width = 900, Height = 600, Content = view };
    Fold.SetProvider(window, new SimulatedFoldStateProvider(
        new FoldState(true, DevicePosture.HalfOpened, 110, new Rect(430, 0, 40, 600), true)));
    window.Show();
    view.UpdateLayout();

    Assert.Equal(TwoPaneViewMode.Wide, view.Mode);
    Assert.Equal(new Rect(0, 0, 430, 600), view.Pane1!.Bounds);
    Assert.Equal(new Rect(470, 0, 430, 600), view.Pane2!.Bounds);
}
```

Avalonia.Headless.XUnit with a simulated provider. The library's own tests work this way.

Then check on the real simulators

Sample 11

iPhone Duo simulator

1. Install Xcode 27.1 with the iOS 27.1 runtime
2. Build the iOS head against the 27.1 SDK
3. Change postures with the buttons in Device Hub

Pixel Fold emulator, API 34

```
# The image has no fold: give it one
adb shell settings put global \
  display_features "fold-[1104,0,1104,1840]"

# 1 closed · 2 half open · 3 open
adb emu posture 2

# Move the hinge
adb emu sensor set hinge-angle0 110

# Read the Sample's output
adb logcat -s DOTNET
```

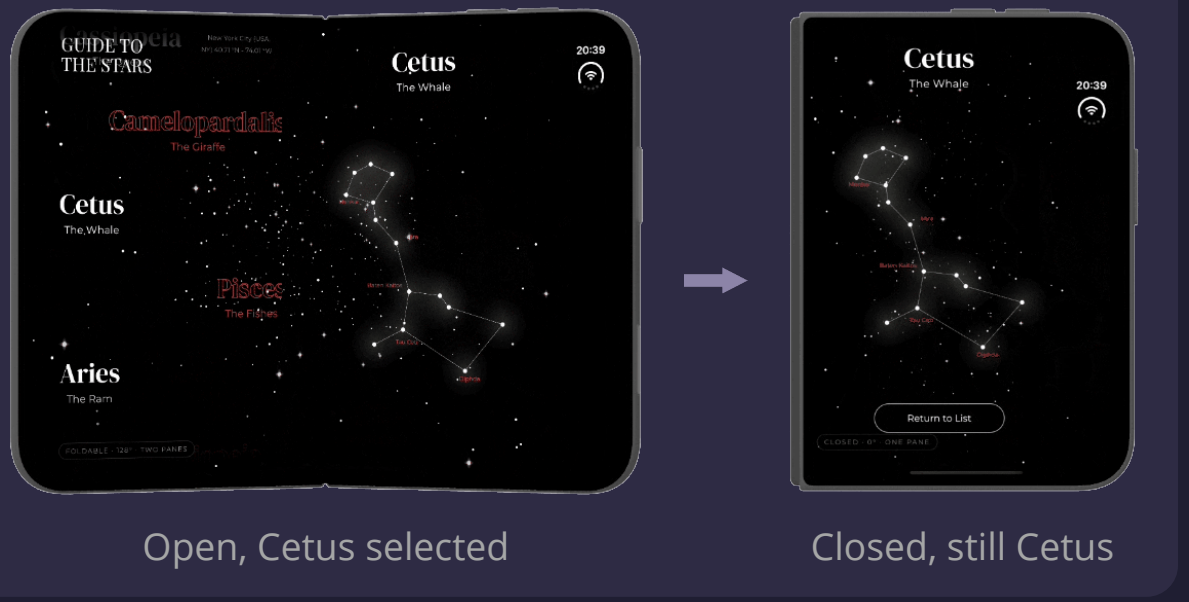
Part 4 of 4 · Tips

A fold is a resize, not a relaunch.

The user was reading, typing and scrolling.
After the fold, they still should be.

Keep the app alive through the fold

Tips



Open, Cetus selected

Closed, still Cetus

1. On Android, handle configuration changes, or folding can recreate the activity.
2. Hide panes, don’t remove them. TwoPaneView keeps the hidden pane alive, state and all.
3. Keep state in view models: selection, drafts, navigation.
4. Never cache screen sizes. Read Bounds when you lay out.

Keep the scroll position

Tips

```
Reader.ScrollChanged += (_, e) ⇒
{
    // Real scrolling only, not a resize
    if (e.ViewportDelta == default && e.ExtentDelta == default)
        _vm.ReadingPosition = CharacterAtTop();
};

// After the fold, bring the same words back to the top
Body.SizeChanged += (_, e) ⇒
{
    if (e.WidthChanged && _vm.ReadingPosition > 0)
        Dispatcher.UIThread.Post(
            () ⇒ ScrollToCharacter(_vm.ReadingPosition));
};
```

Anchor to the words

The width changes, every line rewraps, and the same pixel offset lands in another paragraph.

Lists keep their place

A virtualizing ListBox keeps its first row through a resize.

Keep the focus where the user is

Tips

```
AddHandler(GotFocusEvent, (_, e) ⇒
    (_focused, _restore) = (e.Source as Control, null));
Panes.ModeChanged += (_, _) ⇒
{
    if (_restore is { IsEffectivelyVisible: true } back)
        back.Focus();                       // its pane is back
    else if (_focused is { IsEffectivelyVisible: false } hidden)
    {
        CounterpartOf(hidden)?.Focus();   // land next door
        _restore = hidden;                  // its pane just hid
    }
    else
        return;                             // nothing moved
    _focused?.BringIntoView();
};
```

Never focus the invisible

Move focus next door when a pane hides, and bring it back when the pane returns.

Text survives

Hidden panes keep their text and caret, and their selection with ClearSelectionOnLostFocus off.

Twelve rules to take home

Tips

Space, not device
Let the width pick the layout. Never isTablet.

Clear the crease
No buttons, fields or faces on the fold.

Even columns
Two or four columns split cleanly at the fold.

Ease, don’t jump
Folding should feel like more room, not a new app.

Angles for effects
Lay out by posture and bounds, never the angle.

Map, then measure
FoldBounds is top-level. Safe areas pad content.

Pause hidden panes
Their timers keep running. Check IsEffectivelyVisible.

Pick the one pane
PanePriority decides what the cover display shows.

Back, when alone
Show a back button only in single-pane mode.

Scale, don’t clip
A LayoutTransformControl fits narrow panes.

Mind Split View
On the Duo, your window is not the screen.

Test both ways
Simulated folds in CI, then the real simulators.

Thank you

Now go unfold your UI.

The slides and demos
jsuarezruiz/dotnetconf2026-avalonia-foldables

The library
github.com/jsuarezruiz/Nova.Avalonia.Foldable

The docs
jsuarezruiz.github.io/Nova.Avalonia.Foldable

Guide To the Stars and friends
github.com/jsuarezruiz/avalonia_vignettes

Reports from real devices welcome
#dotnetconf

Javier Suárez · Avalonia UI · @jsuarezruiz

